

DSA Lab — Full Explanation

Big O · Frequency Counter · Multiple Pointers
Sliding Window · Recursion

JavaScript | Beginner-Friendly | Annotated Code

7 Problems · Step-by-Step · Time & Space Complexity

Problems at a Glance

#	Problem	Pattern	Time	Space
1	same()	Frequency Counter	O(n)	O(n)
2	validAnagram()	Frequency Counter	O(n)	O(1)
3	sumZero()	Multiple Pointers	O(n)	O(1)
4	countUniqueValues()	Multiple Pointers	O(n)	O(1)
5	maxSubarraySum()	Sliding Window (Fixed)	O(n)	O(1)
6	minSubArrayLen()	Sliding Window (Dynamic)	O(n)	O(1)
7	findLongestSubstring()	Sliding Window + Freq. Counter	O(n)	O(1)

Problem 1 — same(arr1, arr2)

Pattern: Frequency Counter

What is this problem asking?

Given two arrays, return **true** if every number in the first array has its **square** present in the second array — and both values appear the **same number of times**.

Examples from the lab:

```
same([1,2,3], [4,1,9]) // true → 12=1 ✓ 22=4 ✓ 32=9 ✓  
same([1,2,3], [1,9]) // false → 22=4 is missing  
same([1,2,1], [4,4,1]) // false → 1 appears twice but 12=1 appears once
```

Brute Force Idea ($O(n^2)$ — slow)

For every element in arr1, scan all of arr2 to find its square using **indexOf()** — a loop inside a loop gives $O(n^2)$. Works, but slow for large inputs.

Why Frequency Counter?

Instead of comparing values one by one, we **count how many times each value appears** in each array using a plain object. Then we compare the two counters in one final pass. Three separate loops (each $O(n)$) is far better than nested loops.

Step-by-Step Logic

- If `arr1.length !== arr2.length` → return false immediately.
- Build `counter1` by counting each value in `arr1`.
- Build `counter2` by counting each value in `arr2`.
- For every key in `counter1`, check: (a) does `key2` exist in `counter2`? (b) does the count match?
- If all checks pass → return true.

Metric	Complexity
Time Complexity	$O(n)$
Space Complexity	$O(n)$

Final JavaScript Code

```
function same(arr1, arr2) {  
  // Step 1: If lengths are different, it's impossible to match  
  if (arr1.length !== arr2.length) {  
    return false;  
  }  
  
  // Step 2: Create two empty objects to count frequency of values  
  let counter1 = {};  
  let counter2 = {};
```

```

// Step 3: Count each value in arr1
for (let val of arr1) {
  counter1[val] = (counter1[val] || 0) + 1;
}

// Step 4: Count each value in arr2
for (let val of arr2) {
  counter2[val] = (counter2[val] || 0) + 1;
}

// Step 5: Compare the two counters
for (let key in counter1) {
  if (!(key ** 2 in counter2)) return false;
  if (counter2[key ** 2] !== counter1[key]) return false;
}

return true;
}

```

Key line: `counter1[val] = (counter1[val] || 0) + 1` — if the key doesn't exist yet, treat it as 0 then add 1. After looping `[1,2,1]`: `counter1 = { 1:2, 2:1 }`.

Problem 2 — validAnagram(str1, str2)

Pattern: Frequency Counter

What is this problem asking?

An **anagram** is when you rearrange all letters of one word to form another (e.g. 'cinema' → 'iceman'). The function checks if both strings use the exact same letters, the exact same number of times.

Examples from the lab:

```
validAnagram('', '') // true
validAnagram('aaz', 'zza') // false — 'a' count mismatch
validAnagram('anagram', 'nagaram') // true
validAnagram('rat', 'car') // false — 'c' not in 'rat'
validAnagram('qwerty', 'qeywrt') // true — just shuffled
```

Brute Force ($O(n \log n)$)

Sort both strings and compare. Works! But sorting is $O(n \log n)$. Frequency counter gives us $O(n)$.

The Clever Trick — One Counter, Drain It

Build ONE counter from str1, then loop through str2 and **subtract** from it. If a letter in str2 isn't in the counter (0 or undefined = falsy) → not an anagram. This avoids building two counters and comparing them.

Step-by-Step Logic

- If `str1.length !== str2.length` → return false.
- Build counter from str1 (count each letter).
- Loop through str2: if `counter[letter]` is falsy → return false. Otherwise subtract 1.
- If the loop finishes without issues → return true.

Metric	Complexity
Time Complexity	$O(n)$
Space Complexity	$O(1)$ — at most 26 keys

Final JavaScript Code

```
function validAnagram(str1, str2) {
  // Step 1: Different lengths = impossible to be anagrams
  if (str1.length !== str2.length) {
    return false;
  }

  // Step 2: Build a frequency counter from the FIRST string
  let counter = {};
  for (let letter of str1) {
```

```
counter[letter] = (counter[letter] || 0) + 1;
}

// Step 3: Loop through the SECOND string and drain the counter
for (let letter of str2) {
  if (!counter[letter]) {
    return false; // missing or already used up
  }
  counter[letter]--;
}

return true;
}
```

Why `!counter[letter]` works: Both *undefined* (letter never seen) and *0* (letter used up) are falsy in JavaScript — so one check handles both cases.

Problem 3 — sumZero(arr)

Pattern: Multiple Pointers

What is this problem asking?

Given a **sorted array of integers**, find the **first pair** of numbers that add up to zero. Return them as an array, or **undefined** if no such pair exists.

Examples from the lab:

```
sumZero([-3,-2,-1,0,1,2,3]) // [-3, 3] → -3 + 3 = 0
sumZero([-2,0,1,3]) // undefined → no pair
sumZero([1,2,3]) // undefined → all positive
```

Brute Force ($O(n^2)$)

Two nested loops check every possible pair. Works but slow — $O(n^2)$. The sorted nature of the array lets us do much better.

Why Multiple Pointers (from opposite ends)?

Because the array is **sorted**, the smallest value is on the far left and the largest on the far right. We place one pointer at each end and squeeze inward. At every step we know exactly which direction to move.

Visual Trace

```
arr = [-3, -2, -1, 0, 1, 2, 3]
  ^  ^
left right
Step 1: -3 + 3 = 0 → Found! Return [-3, 3]
arr = [-2, 0, 1, 3]
Step 1: -2 + 3 = 1 → too big, right--
Step 2: -2 + 1 = -1 → too small, left++
Step 3: 0 + 1 = 1 → too big, right--
Step 4: left >= right → return undefined
```

Metric	Complexity
Time Complexity	$O(n)$
Space Complexity	$O(1)$

Final JavaScript Code

```
function sumZero(arr) {
  let left = 0; // points at smallest value
  let right = arr.length - 1; // points at largest value
  while (left < right) {
    let sum = arr[left] + arr[right];
    if (sum === 0) {
```

```
return [arr[left], arr[right]]; // found it!
} else if (sum > 0) {
  right--; // too big → move right pointer left
} else {
  left++; // too small → move left pointer right
}
}

return undefined; // no pair found
}
```

Why $left < right$ (not $\leq$)? If both pointers land on the same index, we'd be adding a number to itself — that's not a valid pair of two different elements.

Problem 4 — countUniqueValues(arr)

Pattern: Multiple Pointers

What is this problem asking?

Given a **sorted array** (may contain negatives), count how many **unique values** are in it. Because the array is sorted, duplicates always sit next to each other.

Examples from the lab:

```
countUniqueValues([1,1,1,1,1,2]) // 2 → {1, 2}
countUniqueValues([1,2,3,4,4,4,7,7,12,12,13]) // 7
countUniqueValues([1]) // 0
countUniqueValues([-2, -1, -1, 0, 1]) // 4 → {-2, -1, 0, 1}
```

Why Multiple Pointers (same direction)?

We use a **slow pointer i** (bookmark for the last confirmed unique value) and a **fast pointer j** (scout ahead). When j finds something different from i, we move i forward. The for loop automatically advances j.

Visual Trace

```
arr = [1, 1, 2, 3, 3]
i i
arr[i]=1 == arr[i]=1 → duplicate, i++
i i
arr[i]=1 != arr[i]=2 → new! i++, i++
i i
arr[i]=1 != arr[i]=2 → (i now at idx 1 val 1, i at idx 2 val 2)
wait – after i++ i=1 still points to 1...
arr[i=1]=1 != arr[i=2]=2 → i++ → i=2, i++
arr[i=2]=2 != arr[i=3]=3 → i++ → i=3, i++
i out of bounds. Answer: i+1 = 4... no wait [1,1,2,3,3] → 3 unique
// Corrected: after loop i=2, answer = 2+1 = 3 unique values ✓
```

Metric	Complexity
Time Complexity	O(n)
Space Complexity	O(1)

Final JavaScript Code

```
function countUniqueValues(arr) {
  // Edge case: empty array
  if (arr.length === 0) return 0;

  let i = 0; // slow pointer – last known unique index
  for (let j = 1; j < arr.length; j++) {
    if (arr[j] !== arr[i]) {
      i++; // found a new unique value, move i forward
    }
  }
  return i + 1;
}
```

```
}  
// if equal, i just keeps moving (for loop handles it)  
}  
return i + 1; // index → count  
}
```

Why return $i+1$? i is an index (0-based). If the last unique value is at index 2, there are 3 unique values (indices 0,1,2).

Bonus: A Set-based solution (`new Set(arr).size`) also works and is perfectly valid — but uses $O(n)$ space vs $O(1)$ here.

Problem 5 — maxSubarraySum(arr, n)

Pattern: Sliding Window (Fixed)

What is this problem asking?

Given an array of integers and a number **n**, find the **maximum sum** of any **n consecutive elements**. The window is always exactly n elements wide.

Examples from the lab:

```
maxSubarraySum([1,2,5,2,8,1,5], 2) // 10 → [2,8]
maxSubarraySum([1,2,5,2,8,1,5], 4) // 17 → [2,5,2,8]
maxSubarraySum([4,2,1,6], 1) // 6 → [6]
maxSubarraySum([], 4) // null → empty array
```

Brute Force ($O(n^2)$)

For every starting position, add up n elements — a loop inside a loop. A lot of repeated addition work.

The Sliding Window Insight

Instead of re-adding n numbers from scratch every step, **reuse the previous window's sum**: subtract the element leaving on the left, add the element entering on the right. One subtraction + one addition per step.

Visual Trace (n=2)

```
arr = [1, 2, 5, 2, 8, 1, 5]
Window [1,2] sum = 3
Window [2,5] sum = 3 - 1 + 5 = 7
Window [5,2] sum = 7 - 2 + 2 = 7
Window [2,8] sum = 7 - 5 + 8 = 10 ← MAX
Window [8,1] sum = 10 - 2 + 1 = 9
Window [1,5] sum = 9 - 8 + 5 = 6
Answer: 10 ✓
```

Metric	Complexity
Time Complexity	$O(n)$
Space Complexity	$O(1)$

Final JavaScript Code

```
function maxSubarraySum(arr, n) {
  if (arr.length < n) return null;
  // Calculate sum of first window manually
  let maxSum = 0;
  for (let i = 0; i < n; i++) {
    maxSum += arr[i];
  }
```

```
let tempSum = maxSum; // current window sum
// Slide the window: start from index n
for (let i = n; i < arr.length; i++) {
  // subtract element leaving left, add element entering right
  tempSum = tempSum - arr[i - n] + arr[i];
  if (tempSum > maxSum) maxSum = tempSum;
}
return maxSum;
}
```

Key formula: $\text{tempSum} - \text{arr}[i - n] + \text{arr}[i]$. When $i=3, n=2$: $\text{arr}[i-n]=\text{arr}[1]$ leaves the window, $\text{arr}[i]=\text{arr}[3]$ enters.

Problem 6 — minSubArrayLen(arr, sum)

Pattern: Sliding Window (Dynamic)

What is this problem asking?

Given an array of **positive integers** and a **target sum**, find the **minimum length** of a contiguous subarray whose sum is $\geq$ **target**. Return 0 if no such subarray exists.

Examples from the lab:

```
minSubArrayLen([2,3,1,2,4,3], 7) // 2 → [4,3]
minSubArrayLen([2,1,6,5,4], 9) // 2 → [5,4]
minSubArrayLen([3,1,7,11,2,9,8,21,62,33,19], 52) // 1 → [62]
minSubArrayLen([1,4,16,22,5,7,8,9,10], 95) // 0 → impossible
```

Fixed vs Dynamic Sliding Window

Problem 5 — Fixed	Problem 6 — Dynamic
Window size is always n	Window grows and shrinks
Only slides → never resizes	Expand when too small, shrink when valid

Strategy: Expand then Shrink

- Move 'end' right → add arr[end] to currentSum (expand).
- While currentSum $\geq$ target: record window length, then shrink from left (start++).
- Repeat until end reaches the end of the array.
- Return 0 if no valid window was found (minLen still Infinity).

Metric	Complexity
Time Complexity	$O(n)$
Space Complexity	$O(1)$

Still $O(n)$ even with two loops! start and end each move at most n steps total across the entire function. start never resets backward.

Final JavaScript Code

```
function minSubArrayLen(arr, sum) {
  let minLen = Infinity; // best length found so far
  let start = 0; // left edge of window
  let end = 0; // right edge of window
  let currentSum = 0; // sum inside current window
  while (end < arr.length) {
    currentSum += arr[end]; // expand: add right element
    // shrink from left while window is still valid
    while (currentSum >= sum) {
      currentSum -= arr[start];
      start++;
      minLen = Math.min(minLen, end - start + 1);
    }
    end++;
  }
  return minLen === Infinity ? 0 : minLen;
}
```

```
while (currentSum >= sum) {  
  let windowLen = end - start + 1;  
  if (windowLen < minLen) minLen = windowLen;  
  currentSum -= arr[start]; // remove left element  
  start++;  
}  
  
end++; // expand: move right edge forward  
}  
  
return minLen === Infinity ? 0 : minLen;  
}
```

Why Infinity as starting value? We're looking for the minimum length. Any real window length (1,2,3...) is smaller than Infinity, so the first valid window automatically becomes our new best.

Problem 7 — findLongestSubstring(str)

Pattern: Sliding Window + Freq. Counter

What is this problem asking?

Given a string, find the **length of the longest substring** where **all characters are distinct** (no repeats). Characters must be consecutive.

Examples from the lab:

```
findLongestSubstring('') // 0
findLongestSubstring('rithmschool') // 7 → 'rithmsc'
findLongestSubstring('thisisawesome') // 6 → 'awesom'
findLongestSubstring('bbbbbb') // 1 → just one 'b'
findLongestSubstring('longestsubstring') // 8 → 'ubstring'
```

Why Two Patterns Together?

The **Sliding Window** defines the boundaries of our current substring (start → end). The **Frequency Counter** (object mapping char → last seen index) tells us instantly if a character is already inside the window — no inner loop needed.

The Key Trick — Store Last-Seen Index (not just count)

When we find a duplicate at position end, we need to jump start directly past the previous occurrence. Storing the **index** instead of a count lets us do this in O(1). Always use `Math.max(start, seen[char]+1)` to never move start backward.

Visual Trace

```
str = 'abcabc'
end=0 (a): not seen → seen={a:0}, window='a', len=1
end=1 (b): not seen → seen={a:0,b:1}, window='ab', len=2
end=2 (c): not seen → seen={a:0,b:1,c:2}, window='abc', len=3
end=3 (a): SEEN at 0, start=max(0,0+1)=1, seen={a:3,...}, window='bca', len=3
end=4 (b): SEEN at 1, start=max(1,1+1)=2, seen={b:4,...}, window='cab', len=3
end=5 (c): SEEN at 2, start=max(2,2+1)=3, seen={c:5,...}, window='abc', len=3
Answer: 3 ✓
```

Metric	Complexity
Time Complexity	O(n)
Space Complexity	O(1) — at most 26 keys

Final JavaScript Code

```
function findLongestSubstring(str) {
  if (str.length === 0) return 0;
  let longest = 0;
```

```
let seen = {}; // maps char → last index where we saw it
let start = 0; // left edge of window
for (let end = 0; end < str.length; end++) {
  let char = str[end];
  // If char is in window, jump start past its old position
  if (seen[char] !== undefined && seen[char] >= start) {
    start = seen[char] + 1;
  }
  seen[char] = end; // always update last-seen index
  let currentLen = end - start + 1;
  if (currentLen > longest) longest = currentLen;
}
return longest;
}
```

Critical check: `seen[char] >= start` — we only care about duplicates that are *inside* the current window. An old occurrence before `start` has already been removed from the window; we must ignore it.

Always update `seen[char] = end` whether it was a duplicate or not — we always want the most recent index for each character.

Complete Summary

#	Function	Pattern	Time	Space	Key Insight
1	same()	Freq. Counter	$O(n)$	$O(n)$	Build 2 counters, compare squared keys
2	validAnagram()	Freq. Counter	$O(n)$	$O(1)$	Build 1 counter, drain with str2
3	sumZero()	Multi Pointers	$O(n)$	$O(1)$	Squeeze from both ends inward
4	countUniqueValues()	Multi Pointers	$O(n)$	$O(1)$	Slow+fast pointer, same direction
5	maxSubarraySum()	Sliding Window Fixed	$O(n)$	$O(1)$	Reuse sum: -left +right
6	minSubArrayLen()	Sliding Window Dynamic	$O(n)$	$O(1)$	Expand then shrink aggressively
7	findLongestSubstring()	Slide + Freq	$O(n)$	$O(1)$	Store last index, jump on duplicate

When to Use Each Pattern

Frequency Counter	Anytime you need to compare two things by what they contain and how many times. Keywords: 'same frequency', 'anagram', 'contains same values'.
Multiple Pointers	Sorted array + looking for a pair or counting something. Opposite ends: finding pairs. Same direction: comparing neighbors.
Sliding Window (Fixed)	Find max/min sum of exactly n consecutive elements. Window size is constant — just slide and update edges.
Sliding Window (Dynamic)	Find shortest/longest subarray meeting a condition. Window grows (expand right) and shrinks (move left) based on validity.

The biggest takeaway from this entire lab: every single problem could be solved with brute-force nested loops in $O(n^2)$. By recognizing the right pattern, every one becomes $O(n)$. That is the whole point of learning these patterns.